



INSTITUTO POLITÉCNICO NACIONAL

**ESCUELA SUPERIOR DE INGENIERÍA MECÁNICA Y
ELÉCTRICA
UNIDAD CULHUACÁN**

TESINA

*Ecosistema multiagente basado en LLM y aprendizaje por
refuerzo para la generación y validación de circuitos analógicos
CA/CD a partir de lenguaje natural*

QUE PARA OBTENER EL TÍTULO DE:

INGENIERO EN COMPUTACIÓN

PRESENTA:

ANTONIO DE JESUS TORRES MARTÍNEZ

DIEGO JESUS GUTIERREZ DELGADO

ASESOR:

RICARDO ISRAEL CALZADA SALAS



Ciudad de México 21 de junio de 2026

Índice general

1. Introducción	1
Planteamiento del Problema	1
Objetivos	2
Justificación	3
Límites y Alcances	4
2. Marco Teórico	5
3. Estado del Arte	6
3.1. Antecedentes Relevantes	6
3.2. Ubicación de este Proyecto	7
4. Diseño e Implementación de la Solución	9
4.1. Introducción	9
4.2. Premisas y fundamento de diseño	10
4.2.1. El problema de diseño y el papel del SRS	10
4.2.2. Premisas de diseño	10
4.3. Descomposición funcional y casos de uso	12
4.3.1. Vista funcional	12
4.3.2. Modelo de casos de uso	13
4.4. Arquitectura del sistema (modelo C4)	14
4.4.1. Nivel 1 — Contexto	14
4.4.2. Nivel 2 — Contenedores	15
4.4.3. Nivel 3 — Componentes	16
4.5. Comportamiento dinámico: flujo de generación	17
4.6. Criterios de aceptación de la propuesta	20
4.7. Síntesis del diseño	20
5. Pruebas y Análisis de Resultados	21
6. Conclusiones y Trabajo Futuro	22
Referencias	23

Índice de figuras

1.1. Diagrama de justificación del ecosistema multiagente.	4
3.1. Posición de los trabajos revisados según cuatro rasgos.	7
4.1. Diagrama funcional del sistema.	13
4.2. Diagrama de casos de uso: flujo de simulación y validación.	14
4.3. C4 Nivel 1: Diagrama de contexto.	15
4.4. C4 Nivel 2: Diagrama de contenedores.	16
4.5. C4 Nivel 3: Diagrama de componentes.	17
4.6. Diagrama de secuencia del flujo de generación.	19
A.1. Diagrama de casos de uso: acceso y gestión de sesiones.	25
A.2. Diagrama de casos de uso: generación de circuito.	26
A.3. Diagrama de casos de uso: decisión del agente curador.	26

Capítulo 1

Introducción

Planteamiento del Problema

A diferencia del diseño digital, que se apoya en abstracciones estandarizadas y flujos de automatización maduros, el diseño de circuitos analógicos descansa en el conocimiento experto, el razonamiento heurístico y un ajuste iterativo guiado por simulación [1]. Ese ciclo —proponer un circuito, simularlo, revisar los resultados y volver a ajustar— se repite numerosas veces antes de obtener un diseño que cumpla las metas, y validar un diseño completo puede exigir largos periodos de iteración [2]. Buena parte de ese conocimiento reside en la experiencia de quien diseña y no en un procedimiento que pueda seguirse paso a paso, lo que vuelve el diseño analógico difícil tanto de automatizar como de aprender.

Los Modelos de Lenguaje Grande (LLM) abrieron la posibilidad de describir un circuito en lenguaje natural y recibir una propuesta de diseño de forma automática. Al llevarla a la práctica, sin embargo, aparecen limitaciones documentadas. Estos modelos alucinan componentes, violan restricciones físicas del circuito y producen salidas eléctricamente inconsistentes [3], por lo que sus diseños rara vez funcionan en el primer intento, ya que el modelo no comprueba por sí mismo si lo que generó cumple. La respuesta de la investigación ante esto ha sido sustituir el modelo único por varios agentes que se reparten el trabajo, acompañados de un mecanismo que verifica cada propuesta antes de darla por válida.

Los trabajos publicados se reparten entre dos líneas. La primera diseña a nivel de transistor, atado a un proceso de fabricación específico descrito por su *Process Design Kit* (PDK), y emplea la simulación SPICE como realimentación durante el diseño; es la línea más desarrollada [1], [4], [5]. La segunda trabaja a nivel de placa, con componentes comerciales, y verifica el resultado mediante reglas eléctricas [2], [3], en parte porque no todo componente comercial cuenta con un modelo SPICE utilizable, lo que impide simular cualquier diseño de forma uniforme. La simulación SPICE a nivel de placa ha comenzado a aparecer, pero en forma de banco de prueba que compara el diseño generado contra un circuito de referencia construido por expertos [6], y no como una señal que el propio sistema utilice para ajustar el diseño durante su ejecución. Entre los trabajos revisados no se encontró un sistema generativo

de varios agentes que use la simulación SPICE como realimentación dentro del lazo de diseño, con un curador que decida el ajuste, orientado a circuitos analógicos CA/CD con componentes comerciales. Ese es el punto del espacio de diseño que este proyecto se propone ocupar.

De lo anterior se desprende la pregunta que ordena el trabajo:

¿Es posible construir un ecosistema de varios agentes que, a partir de una descripción en lenguaje natural, genere un circuito analógico CA/CD con componentes comerciales, lo simule en ngspice y ajuste sus valores de forma automática hasta cumplir las metas de diseño? ¿Y cómo se comporta su desempeño frente al de un enfoque de un solo modelo que repite el intento varias veces?

Objetivos

Objetivo General

Desarrollar un ecosistema multiagente basado en LLM y aprendizaje por refuerzo para la generación y validación de circuitos analógicos CA/CD con componentes comerciales a partir de descripciones en lenguaje natural, empleando la simulación en ngspice como mecanismo de validación.

Objetivos Específicos

1. **Diseñar** la arquitectura del sistema en dos subsistemas independientes: un cliente que gestiona el acceso de los usuarios y un servidor sin estado que aloja a los agentes, condición que habilita su escalado horizontal.
2. **Desarrollar** el agente orquestador y el patrón Master-Worker que descompone un circuito de varias etapas en sub-bloques y reparte su cálculo entre varios trabajadores.
3. **Implementar** el agente de cálculo, que obtiene los valores de los componentes de cada sub-bloque a partir de las ecuaciones de diseño correspondientes.
4. **Desarrollar** el agente curador, basado en aprendizaje por refuerzo, que decide si una propuesta de circuito se acepta, se ajusta o se rechaza con base en las métricas devueltas por la simulación.
5. **Implementar** la integración con ngspice mediante PySpice, cerrando el lazo de realimentación que permite ajustar los valores de los componentes de forma iterativa.
6. **Evaluar** el desempeño del sistema sobre un conjunto de circuitos de prueba con solución de referencia, mediante el error porcentual absoluto medio (MAPE) y la tasa de circuitos

que cumplen las metas, y contrastarlo con el de un enfoque de un solo modelo que repite el intento varias veces (*best-of-N*).

Justificación

Automatizar el diseño de circuitos analógicos tiene valor práctico porque su ciclo de propuesta, simulación y ajuste descansa en el conocimiento experto y no puede seguirse como un procedimiento mecánico. Los modelos de lenguaje que generan diseños a partir de lenguaje natural simplifican la entrada al proceso, pero producen diseños eléctricamente inconsistentes que requieren corrección manual o, cuando los sistemas verifican internamente, lo hacen a nivel de transistor o mediante reglas eléctricas, no mediante simulación SPICE dentro del lazo de diseño.

En la práctica, a partir de una descripción en lenguaje natural el sistema genera un netlist con componentes comerciales, lo simula en ngspice [7], extrae sus métricas eléctricas y ajusta los valores de los componentes de forma iterativa hasta que el diseño cumpla las metas especificadas. El ciclo de ajuste lo ejecuta el propio ecosistema; el usuario recibe el netlist simulado junto con las métricas y el historial de iteraciones.

Los destinatarios directos son ingenieros y estudiantes que trabajen con circuitos analógicos CA/CD contruidos a partir de componentes comerciales con modelo SPICE disponible —resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales de catálogo— y que requieran una herramienta que no solo genere sino que también valide mediante simulación, sin depender de un nodo tecnológico propietario ni de herramientas de licencia costosa.

La ventaja frente a otros proyectos del área está en el punto que ocupa: trabajar con componentes comerciales y, a la vez, usar la simulación SPICE como señal de ajuste dentro del lazo de diseño, coordinado por varios agentes con un curador. Ninguno de los trabajos revisados reúne esa combinación; el estado del arte desarrolla la comparación en detalle.

El proyecto es factible dentro del plazo de una tesina porque se apoya en herramientas de uso libre —LangGraph [8], PySpice [9] y ngspice [7]—, desplegables en la nube y sin dependencia de licencias. Desde la ingeniería en computación, su aportación consiste en integrar técnicas de varios dominios —arquitectura de software, sistemas multiagente, simulación de circuitos y aprendizaje por refuerzo— en un sistema funcional y en evaluarlo de forma empírica frente a una línea base.

La Figura 1.1 resume este desplazamiento: del flujo manual guiado por el experto y de la generación sin validación de los modelos de lazo abierto, hacia un lazo de validación y ajuste automático.



Figura 1.1. Diagrama de justificación del ecosistema multiagente.

Límites y Alcances

El sistema abarca el recorrido completo desde una descripción en lenguaje natural hasta un netlist simulado en ngspice. Del lado del cliente, gestiona el registro y el inicio de sesión, los espacios de trabajo y las claves con las que el usuario envía sus solicitudes. Del lado del servidor, lee la descripción y extrae de ella los parámetros eléctricos, divide los circuitos de varias etapas en sub-bloques, calcula los valores de los componentes, genera el código PySpice y el netlist, simula en ngspice, lee las métricas, ajusta los valores de forma iterativa a través del curador y entrega al usuario el netlist final junto con sus métricas y el historial de iteraciones.

El sistema trabaja únicamente con circuitos analógicos CA/CD formados por componentes comerciales que admiten un modelo SPICE, como resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales descritos por su macromodelo. Un componente sin un modelo utilizable queda fuera de su alcance, ya que sin modelo no hay simulación y, por tanto, no hay señal de realimentación. El sistema no diseña la disposición física de la placa (PCB), no realiza síntesis a nivel de transistor para un proceso de fabricación específico (PDK) ni genera esquemáticos gráficos. Por último, los valores numéricos que aparecen en los requisitos son metas de la primera versión y pueden ajustarse durante la evaluación experimental.

Capítulo 2

Marco Teórico

Capítulo 3

Estado del Arte

3.1. Antecedentes Relevantes

La automatización y el diseño generativo de circuitos analógicos mediante inteligencia artificial han cobrado especial relevancia en años recientes. Para situar el aporte de este trabajo, resulta indispensable analizar las metodologías propuestas en la literatura técnica reciente.

En primer lugar, **AnalogCoder** [4] introdujo la propuesta de diseñar circuitos tratándolos como código ejecutable; en lugar de pedirle al modelo de lenguaje que describa un circuito en lenguaje natural, se le instruye para que escriba un script en Python capaz de generarlo. Al convertir el diseño en un programa ejecutable se reduce significativamente la ambigüedad de la síntesis y el netlist resultante puede simularse de inmediato. El sistema funciona sin necesidad de reentrenar el modelo base y comprueba recursivamente cada propuesta con una secuencia de verificaciones por simulación SPICE, validando que el circuito simule sin nodos flotantes, que los puntos de operación en corriente continua sean correctos y que la respuesta dinámica o de frecuencia cumpla con las especificaciones. Cuando una verificación falla, la traza de error regresa al modelo para orientar el siguiente intento de corrección. No obstante, esta aproximación coordina todo el flujo con un solo agente y se limita a nivel de transistor sobre un proceso de fabricación específico.

Por su parte, **AnalogAgent** [1] reparte esta carga de trabajo mediante un esquema de colaboración multiagente. Su contribución central consiste en la gestión y destilación del conocimiento acumulado durante los fallos: cuando un intento de diseño resulta fallido, el sistema procesa el error para traducirlo en una regla heurística explícita que se incorpora a una memoria auto-evolutiva y se recupera en iteraciones futuras, reduciendo el número total de llamadas al modelo. Adicionalmente, el estudio documenta una limitación típica de los agentes monolíticos basados en modelos de lenguaje: a medida que el historial de chat crece con cada intento de corrección, el resumen de contexto tiende a diluir u omitir detalles e instrucciones técnicas críticas. Esta estrategia de acumular conocimiento para guiar los pasos subsiguientes influye directamente en la concepción del agente curador de este trabajo, aunque mantiene la limitación de operar a nivel de transistor.

Finalmente, **CircuitLM** [3] comparte con este proyecto el nivel de abstracción a nivel de componentes y módulos comerciales a partir de especificaciones de lenguaje natural. Este sistema emplea una tubería multiagente dedicada a la identificación de componentes, la consulta de hojas de especificaciones y la estructuración del esquema. No obstante, se orienta a electrónica digital y embebida —microcontroladores como Arduino o ESP32— y difiere del presente trabajo en su método de validación: en lugar de integrar simulación SPICE en el lazo, verifica la topología con un motor determinista de reglas eléctricas y un modelo de lenguaje supervisor. Sus propios autores señalan la verificación con simulador en el lazo como una limitación pendiente, justamente la señal de realimentación que este proyecto incorpora.

3.2. Ubicación de este Proyecto

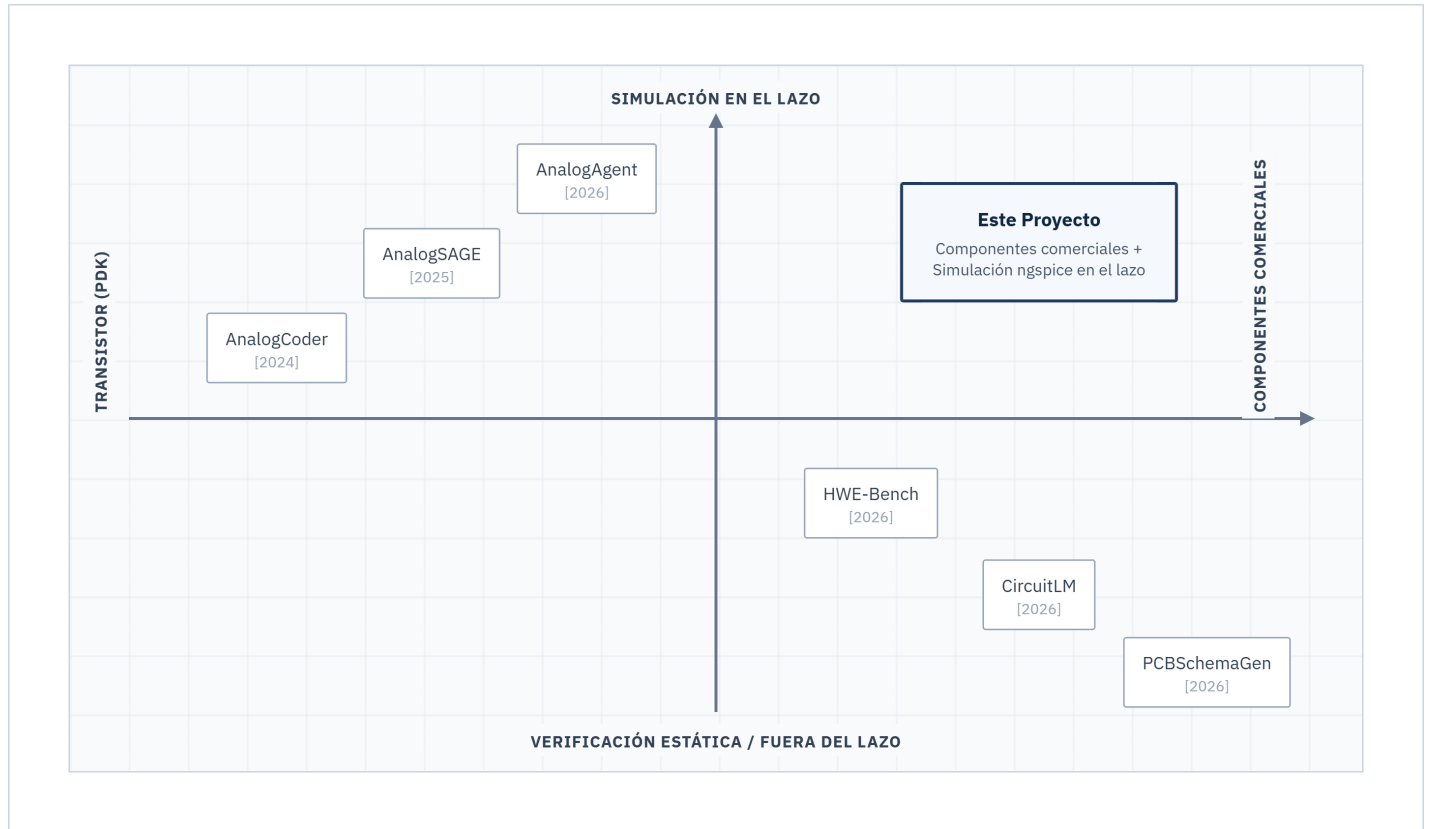


Figura 3.1. Posición de los trabajos revisados según cuatro rasgos.

Junto a los desarrollos mencionados, otras propuestas complementan el panorama del área. **AnalogSAGE** [5] introduce memoria estratificada y mecanismos de autoevolución a la síntesis a nivel de transistor; **PCBSchemaGen** [2] extiende la verificación estática de reglas al ámbito de tarjetas de circuito impreso reales; y **HWE-Bench** [6] provee un entorno

de evaluación comparativa que mide el rendimiento de netlists generados contrastándolos con diseños de referencia de expertos a través de simulaciones SPICE. Adicionalmente, se registran contribuciones como **ADO-LLM** [10] para la optimización bayesiana en el diseño físico y **SPICEPilot** [11] para la generación asistida de scripts de control.

La Figura 3.1 organiza los trabajos según los dos ejes que mejor los distinguen: el nivel de abstracción del diseño —de transistor a componentes comerciales— y la función de la simulación en el flujo —de la verificación estática a la realimentación en el lazo—. El esquema de coordinación y el mecanismo de optimización paramétrica complementan el contraste y se desarrollan en el texto.

Al contrastar las soluciones se observa que ninguno de los antecedentes recopilados integra simultáneamente los cuatro rasgos propuestos en este ecosistema. Mientras que los sistemas orientados a transistores explotan la simulación en lazo cerrado y el ajuste paramétrico iterativo, no contemplan el uso de componentes comerciales. Por el contrario, los enfoques comerciales evitan la simulación dinámica en el lazo de optimización o la reducen a una métrica de prueba externa. Este proyecto une la retroalimentación física por simulación de los flujos a nivel de transistor con el diseño a nivel de placa mediante componentes comerciales, implementando una arquitectura multiagente coordinada por un curador entrenado por aprendizaje por refuerzo.

Nombre del Paper	Origen	Descripción	Enlace
AnalogCoder	 HK / EE. UU.	Primer agente LLM sin entrenamiento para diseño de circuitos analógicos mediante generación de código Python y bibliotecas de subcircuitos.	[Link]
AnalogAgent	 Singapur	Arquitectura multi-agente con memoria auto-evolutiva que aprende de los fallos de simulación sin depender de bibliotecas externas.	[Link]
CircuitLM	 Bangladesh	Tubería multiagente que genera esquemas a nivel de placa con componentes comerciales, orientada a electrónica digital y embebida; valida con un motor de reglas eléctricas (ERC) y un modelo supervisor, sin simulación en el lazo.	[Link]

Tabla 3.1. Resumen - Estado del Arte

Capítulo 4

Diseño e Implementación de la Solución

4.1. Introducción

En el capítulo anterior se revisó el estado del arte de los sistemas multiagente basados en LLM para el diseño de circuitos analógicos, y se identificó que los trabajos reportados operan a nivel de transistor sobre un nodo tecnológico (PDK), mientras que la generación y validación de circuitos a nivel de componentes comerciales (COTS) con simulación SPICE dentro del lazo de diseño permanece como una brecha abierta. Este capítulo presenta la propuesta de solución que atiende esa brecha.

El propósito del capítulo es exponer el diseño del sistema antes de su construcción: las premisas que lo fundamentan, su descomposición funcional, su arquitectura estructural y su comportamiento en ejecución. Las decisiones de diseño se justifican a partir de la naturaleza del problema, de los principios de la ingeniería de software y de los hallazgos del estado del arte; los artefactos de análisis y diseño elaborados a lo largo del proyecto —la Especificación de Requisitos de Software (SRS), el modelo de casos de uso, el diagrama funcional, el modelo arquitectónico C4 de tres niveles y el diagrama de secuencia— registran y representan esas decisiones, de modo que cada una quede expresada en un artefacto verificable.

La exposición sigue un orden de lo abstracto a lo concreto. Primero se establecen el problema que el diseño debe resolver, las premisas que lo orientan y el papel del SRS como registro de los requisitos (sección 4.2). A continuación se describe *qué* hace el sistema mediante su descomposición funcional y sus casos de uso (sección 4.3). Después se describe *cómo* se estructura, mediante el modelo C4 en sus tres niveles (sección 4.4). Enseguida se describe *cómo se comporta* en ejecución, mediante el diagrama de secuencia del flujo principal de generación (sección 4.5). Finalmente se enuncian los criterios de aceptación que la propuesta deberá satisfacer (sección 4.6) y se cierra con una síntesis del diseño y su conexión con la evaluación del sistema (sección 4.7).

4.2. Premisas y fundamento de diseño

4.2.1. El problema de diseño y el papel del SRS

El diseño responde a un problema concreto: transformar la descripción de un circuito analógico CA/CD, expresada en lenguaje natural, en un netlist que haya sido simulado y validado, empleando componentes comerciales en lugar de dispositivos a nivel de transistor. Este problema impone por sí mismo varias exigencias al diseño —la interpretación de lenguaje natural, el cálculo de valores de componentes, la simulación física y el ajuste del circuito hasta cumplir una meta— que son las que, en última instancia, justifican la forma del sistema. Las decisiones de diseño de este capítulo se fundamentan en esas exigencias del problema, en los principios de la ingeniería de software y en los hallazgos del estado del arte, y no en la conformidad con un documento interno.

La Especificación de Requisitos de Software (SRS) en su versión 1.0 cumple un papel distinto al de fuente de justificación: es el artefacto que registra de forma ordenada y verificable los requisitos que el sistema debe satisfacer, los traduce a criterios de verificación y delimita el alcance. El SRS fija qué se construye —el alcance, los requisitos funcionales por subsistema, los requisitos no funcionales y los criterios de aceptación—; las razones por las que el diseño adopta una u otra forma se exponen en este capítulo a partir del problema y de la disciplina, y los identificadores de requisito (RF, RNF) se citan únicamente para señalar qué requisito queda cubierto por cada decisión, no para sustentarla.

El alcance del sistema comprende recibir la descripción de un circuito analógico en lenguaje natural, dividir circuitos de varias etapas en sub-bloques, calcular los valores de los componentes, generar el código PySpice y el netlist, simular en ngspice, ajustar iterativamente los valores mediante un agente curador, y entregar al usuario el netlist final junto con las métricas y el historial de iteraciones. Quedan fuera del alcance el diseño de *layout* físico (PCB), la síntesis a nivel de transistor para un nodo tecnológico y la generación de esquemáticos gráficos.

4.2.2. Premisas de diseño

A partir del problema descrito, y guiado por principios reconocidos de la ingeniería de software, el diseño adopta un conjunto de premisas que orientan las decisiones arquitectónicas de las secciones siguientes. Para cada una se expone primero su fundamento y, después, el requisito del SRS que queda cubierto.

La primera premisa es la separación del sistema en dos subsistemas que se despliegan por separado. Esta decisión aplica el principio de separación de intereses: las responsabilidades

de acceso —autenticación, sesiones, espacios de trabajo y emisión de API keys— pertenecen a un dominio distinto del de la generación de circuitos, y mantenerlas en subsistemas separados reduce el acoplamiento entre ambos y permite desplegarlos, escalarlos y razonar sobre ellos de forma independiente **Pressman**. De ahí que el subsistema cliente actúe como puerta de entrada de los usuarios y no ejecute lógica de diseño de circuitos, que reside íntegramente en el subsistema servidor.

Dentro del servidor, el diseño se rige por la modularidad de los agentes, que materializa los principios de alta cohesión y bajo acoplamiento: cada agente concentra una responsabilidad única y se comunica con los demás solo a través de un estado compartido, sin conocer su implementación interna. Esta organización permite reemplazar o modificar un agente sin alterar los restantes, propiedad que la literatura sobre sistemas multiagente para diseño de circuitos asocia con la capacidad de evolucionar y depurar cada agente por separado, y que en el SRS queda recogida como requisito de mantenibilidad (RNF-04.1).

A esta modularidad se suma la concurrencia en el cálculo, que no es una decisión arbitraria sino una consecuencia de la estructura del problema: un circuito de varias etapas se descompone en sub-bloques, y aquellos que no dependen entre sí pueden calcularse de forma simultánea sin afectar el resultado. El diseño aprovecha esta independencia mediante un patrón Master-Worker con *Map-Reduce*, en el que un nodo Master reparte cada sub-bloque independiente como tarea a un worker y luego recombina los resultados. Esta correspondencia entre la naturaleza paralelizable del problema y la estructura concurrente del sistema queda cubierta por los requisitos de reparto y desempeño del SRS (RF-01.3, RNF-01.2).

El rasgo que distingue la propuesta frente a los enfoques revisados en el estado del arte es la validación dentro del lazo de diseño. Los sistemas multiagente reportados para diseño analógico tienden a emplear la simulación como comprobación posterior a la generación; la propuesta, en cambio, la incorpora como parte del proceso de ajuste: el agente curador compara las métricas obtenidas de ngspice con las metas y decide si el circuito se acepta o se vuelve a ajustar, hasta cumplir la meta o alcanzar un número máximo de iteraciones. Esta integración de la simulación en el lazo es la traducción, al nivel de componentes comerciales, del cierre de realimentación que la literatura sobre diseño asistido por LLM ha mostrado eficaz al nivel de transistor, y constituye la aportación central de la propuesta; en el SRS se recoge como el control del ciclo de ajuste (RF-01.4).

Sostener ese lazo exige conservar el estado entre iteraciones, por lo que el diseño conserva de forma explícita el estado de la generación mediante un mecanismo de *checkpoint* sobre una base de datos, de modo que una solicitud en proceso mantenga su estado disponible para todos los agentes a lo largo de las iteraciones. Esta persistencia se modela de manera explícita en el comportamiento dinámico del sistema (sección 4.5) por ser una

propiedad esencial del lazo, y no una nota accesoria. Finalmente, y en línea con el principio de configurabilidad sin recompilación, los parámetros del agente curador —los pesos, factores y umbrales del cálculo de recompensa, junto con el número máximo de iteraciones— se mantienen en un archivo de configuración externo, de modo que puedan ajustarse durante la experimentación sin modificar el código (RNF-04.2).

4.3. Descomposición funcional y casos de uso

4.3.1. Vista funcional

El sistema ofrece un conjunto acotado de funciones que se desprenden directamente del problema y quedan registradas en el SRS: dar acceso a los usuarios y gestionar sus sesiones, espacios de trabajo y API keys (cliente); leer una descripción de circuito en lenguaje natural y extraer de ella los parámetros eléctricos; dividir un circuito de varias etapas en sub-bloques; calcular los valores de los componentes de cada sub-bloque; generar el código PySpice y el netlist; simular el netlist en ngspice y leer las métricas; comparar las métricas con las metas y decidir si el circuito se acepta o se ajusta; y entregar al usuario el netlist final junto con las métricas y el historial de iteraciones.

El diagrama funcional organiza estas funciones según el subsistema responsable y el flujo de datos que las relaciona, lo que permite ver la propuesta como un encadenamiento de transformaciones sobre la descripción del circuito, desde el lenguaje natural hasta el netlist validado.

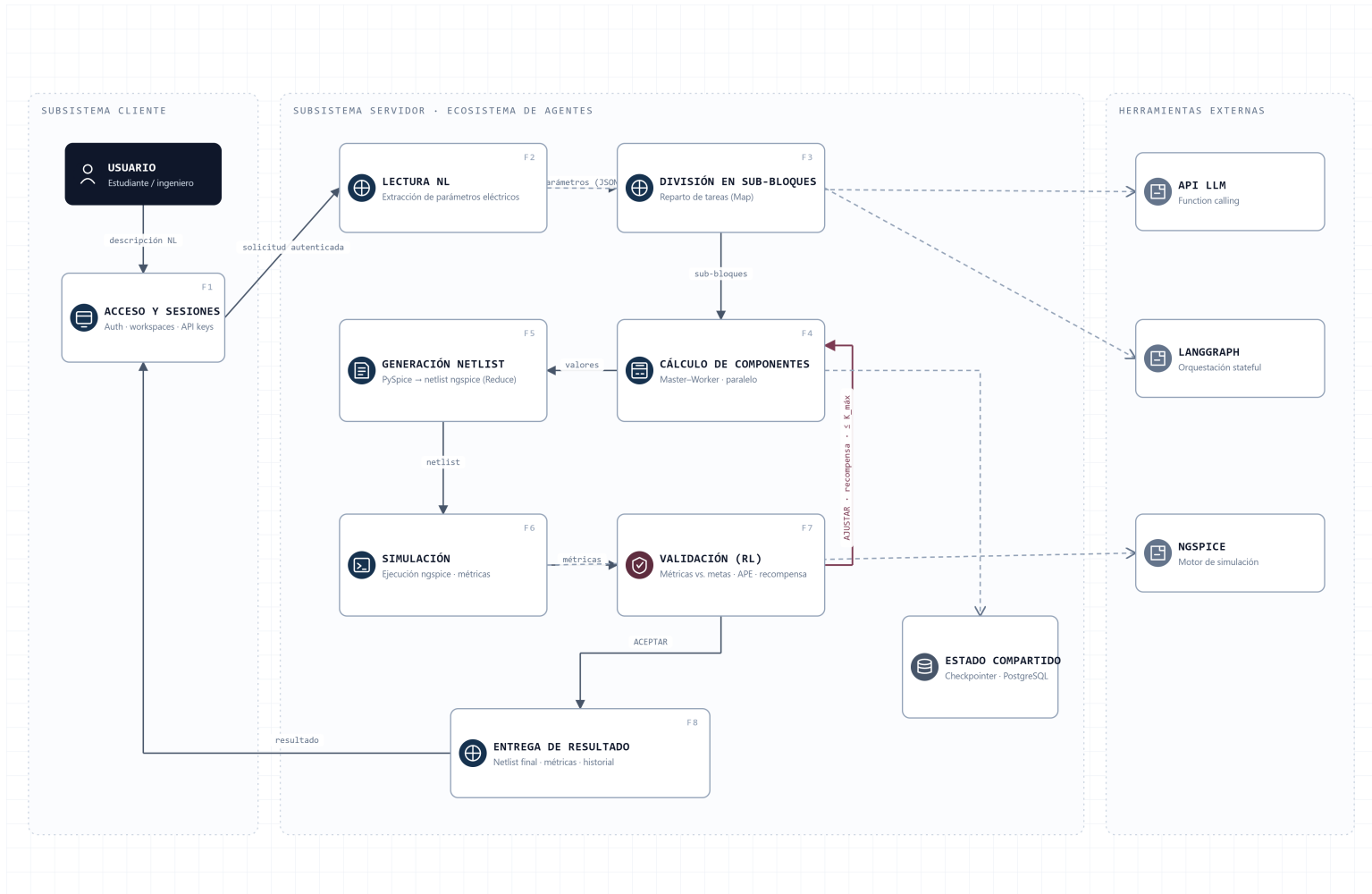


Figura 4.1. Diagrama funcional del sistema.

4.3.2. Modelo de casos de uso

Los casos de uso describen las interacciones que los actores ejecutan sobre el sistema. Se identifican dos clases de actor: el **usuario**, que interactúa con el subsistema cliente, y los **servicios externos** (la API del LLM y el motor ngspice), que el subsistema servidor consume durante la generación.

Los casos de uso principales del usuario incluyen: registrarse e iniciar sesión, gestionar espacios de trabajo, emitir y revocar API keys, y enviar una solicitud de generación de circuito junto con su descripción en lenguaje natural. El caso de uso central —*generar circuito*— desencadena el flujo del subsistema servidor, que se detalla en su comportamiento dinámico en (sección 4.5). Los diagramas de casos de uso restantes (acceso, generación y decisión) se incluyen en el apéndice A.

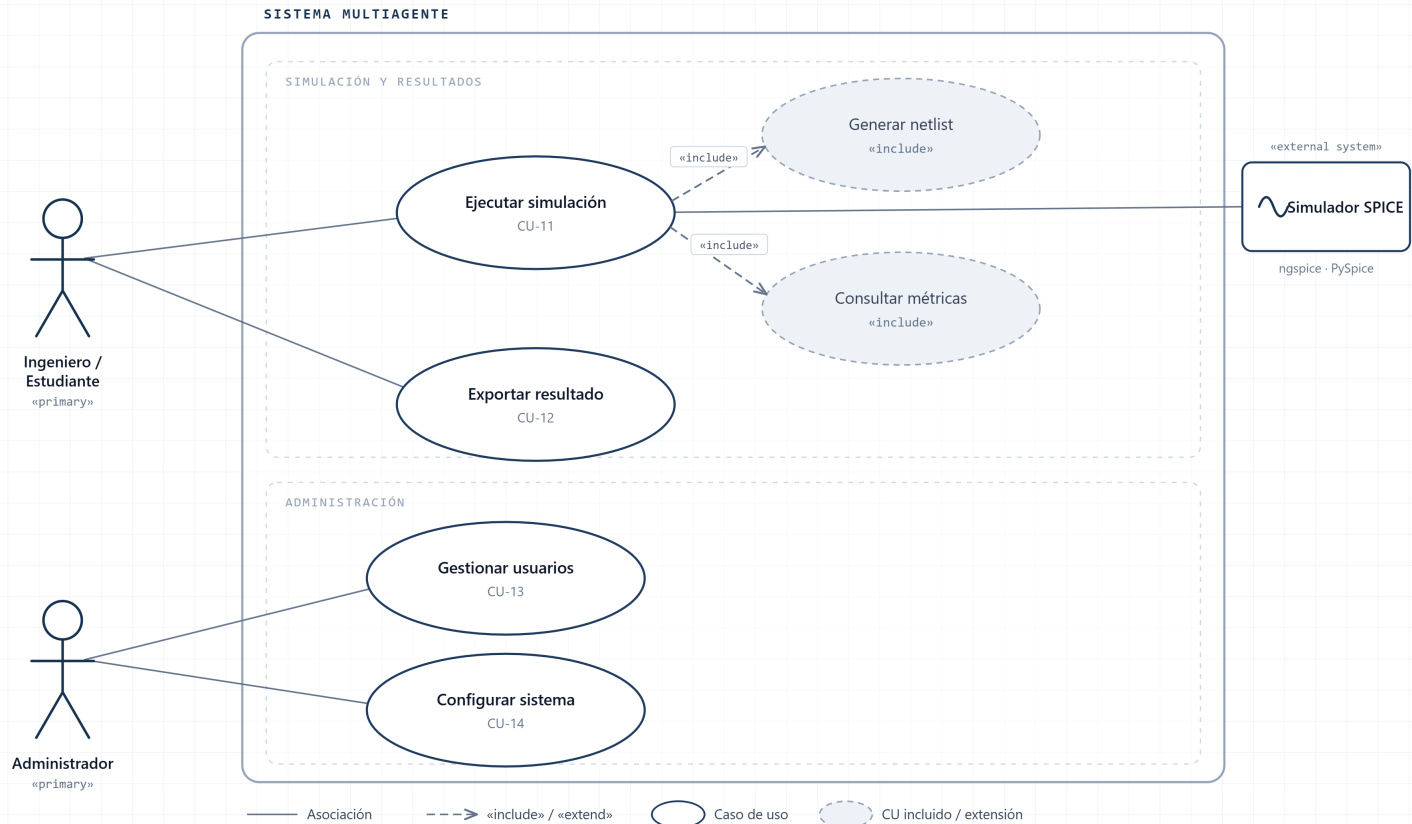


Figura 4.2. Diagrama de casos de uso: flujo de simulación y validación.

4.4. Arquitectura del sistema (modelo C4)

4.4.1. Nivel 1 — Contexto

El nivel de contexto sitúa el sistema frente a sus actores y sistemas externos. Muestra al usuario interactuando con el sistema, y al sistema consumiendo dos servicios externos: la API del LLM y el motor de simulación ngspice. Este nivel deja claro que el sistema es una unidad que media entre la intención del usuario, expresada en lenguaje natural, y dos capacidades externas: el razonamiento del LLM y la simulación física del circuito.

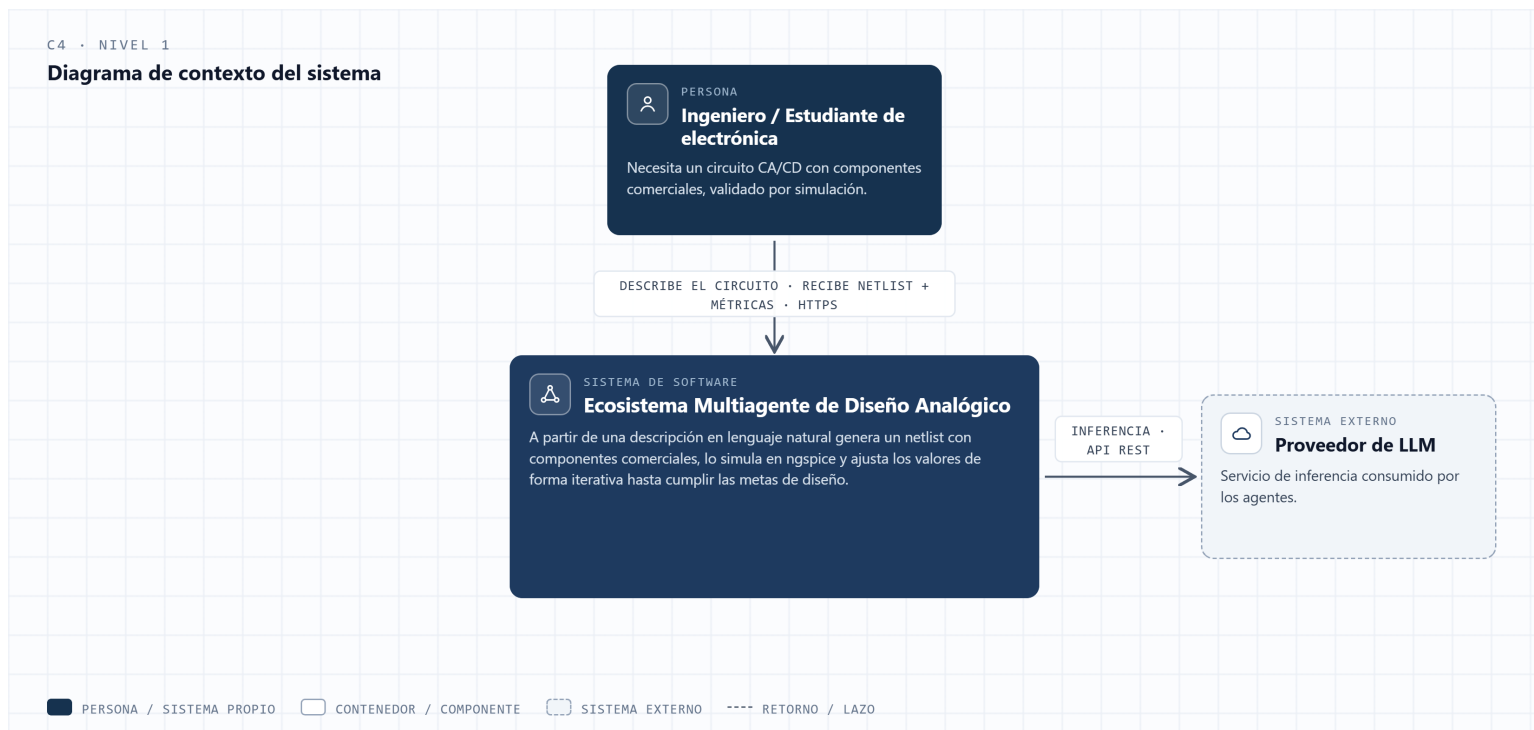


Figura 4.3. C4 Nivel 1: Diagrama de contexto.

4.4.2. Nivel 2 — Contenedores

El nivel de contenedores descompone el sistema en sus unidades desplegables. Hace explícita la separación cliente/servidor: el subsistema cliente (autenticación, sesiones, espacios de trabajo, API keys) y el subsistema servidor (el ecosistema de agentes), junto con la base de datos de persistencia y la frontera con los servicios externos. Este nivel justifica la premisa de despliegue independiente (sección 4.2.2) y muestra el camino de una solicitud autenticada desde el cliente hacia el servidor.

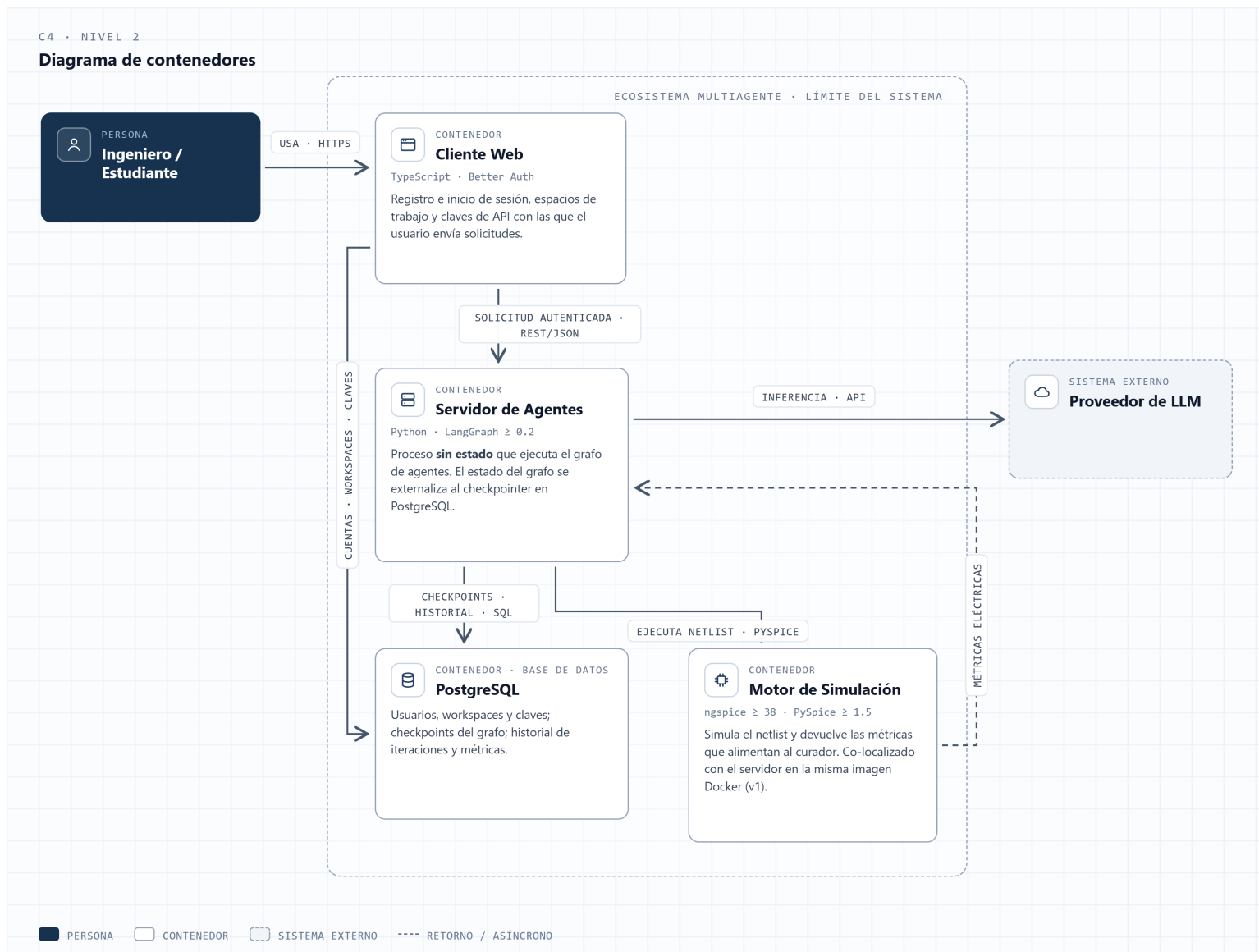


Figura 4.4. C4 Nivel 2: Diagrama de contenedores.

4.4.3. Nivel 3 — Componentes

El nivel de componentes abre el subsistema servidor y muestra el ecosistema de agentes: el agente Orquestador (entrada del servidor y coordinador del estado compartido), el agente de Cálculo (Master-Worker), el agente Curador (RL, lazo de validación iterativa), el agente de Shell (ejecución de ngspice) y el agente de Escritura (generación del netlist con PySpice), todos coordinados sobre el grafo de orquestación con persistencia en base de datos. Este nivel materializa la premisa de modularidad de agentes (sección 4.2.2) y prepara el terreno para la descripción del comportamiento dinámico.

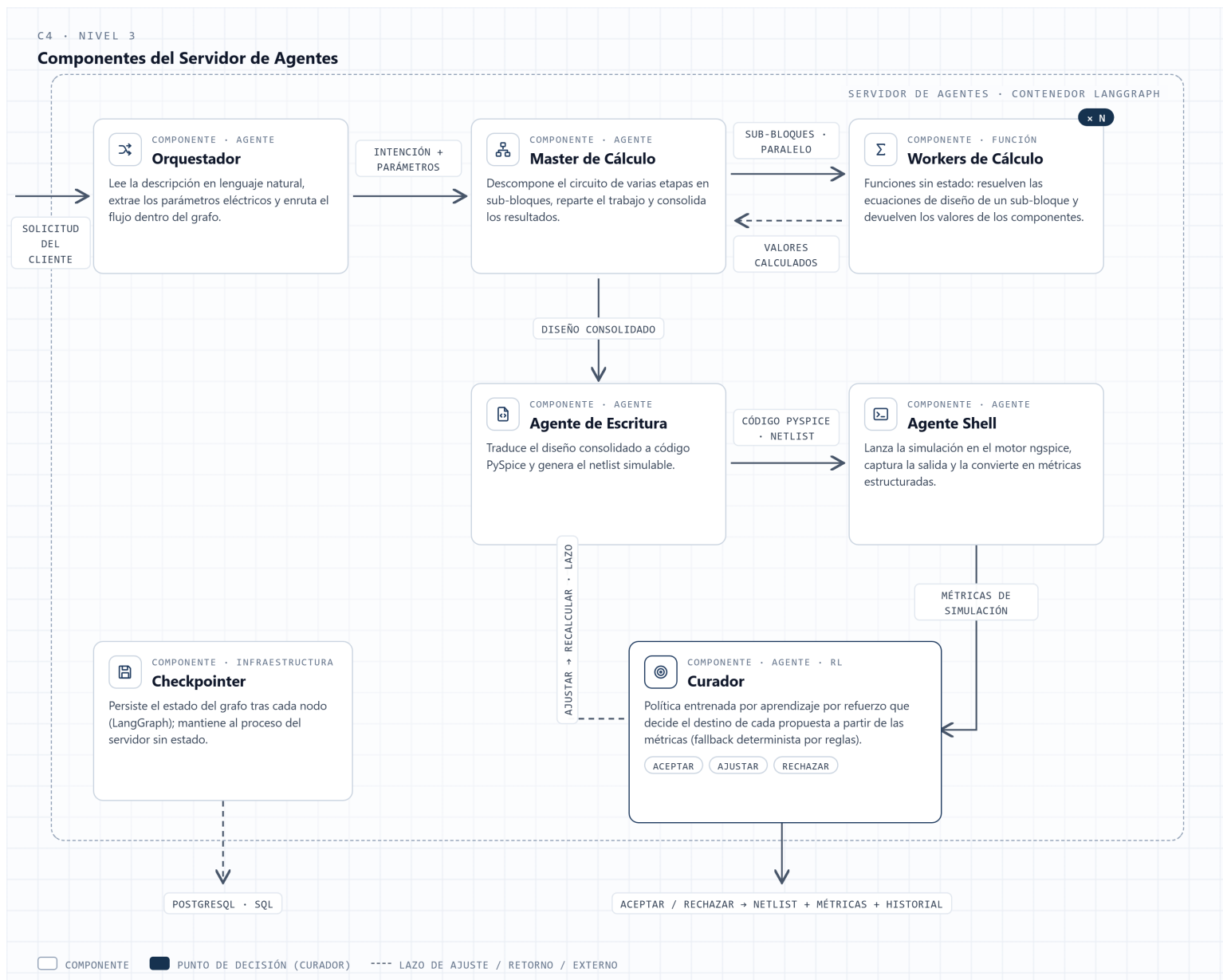


Figura 4.5. C4 Nivel 3: Diagrama de componentes.

4.5. Comportamiento dinámico: flujo de generación

Las secciones anteriores describen la estructura estática del sistema. Esta sección describe su comportamiento en ejecución: cómo colaboran los agentes en el tiempo para transformar una descripción en lenguaje natural en un netlist validado. El diagrama de secuencia integra las vistas anteriores en una línea temporal.

El flujo principal procede así: el orquestador recibe la solicitud autenticada y la descripción del circuito, usa el LLM para extraer los parámetros y la lista de sub-bloques,

y reparte los sub-bloques independientes en paralelo al agente de cálculo (Master-Worker). El agente de escritura genera el código PySpice y el netlist; el agente de shell lo simula en ngspice y lee las métricas; el agente curador compara las métricas con las metas y decide. Según su decisión, el ciclo se repite —ajustando los valores— o termina. El ciclo está acotado por un número máximo de iteraciones configurable (parámetro de guarda del lazo).

El comportamiento de persistencia se modela de forma explícita: el estado de cada iteración se conserva mediante el *checkpoint*, lo que se representa como un fragmento de referencia en el diagrama, de modo que la naturaleza con estado del sistema quede formalmente reflejada y no como una observación accesorio.

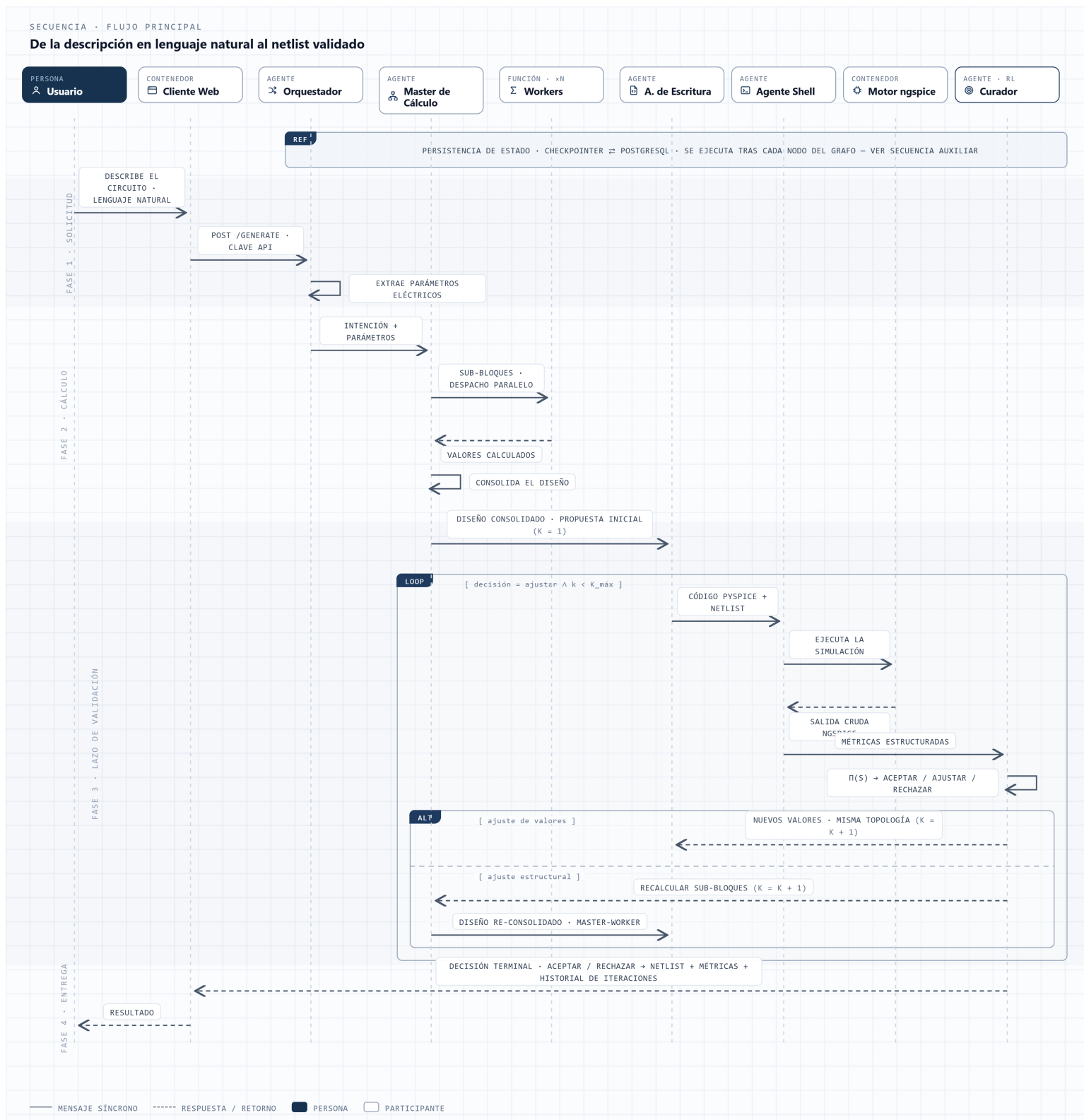


Figura 4.6. Diagrama de secuencia del flujo de generación.

4.6. Criterios de aceptación de la propuesta

La propuesta de solución se considerará satisfactoria si el sistema construido cumple un conjunto de criterios de aceptación objetivos y comprobables de forma independiente. Estos criterios están enunciados en el SRS junto con su procedimiento de verificación, lo que permite contrastarlos con mediciones reales en lugar de con el juicio del diseñador. No se evalúan en este capítulo —su comprobación empírica corresponde al capítulo de experimentos y resultados— pero se enuncian aquí como cierre del diseño, para dejar explícito qué deberá medirse y bajo qué condición se dará por cumplido.

Entre los criterios relevantes figuran: que el sistema acepte una descripción en lenguaje natural y extraiga de ella correctamente los parámetros eléctricos; que ejecute sub-bloques independientes en paralelo; que el lazo de ajuste itere hasta cumplir la meta o alcanzar el máximo de iteraciones; que produzca un netlist simulado en ngspice; y que la salida sea repetible con la temperatura del LLM fijada, para permitir la comparación de resultados durante la evaluación.

4.7. Síntesis del diseño

La propuesta de solución descrita en este capítulo da forma concreta a la brecha identificada en el estado del arte: un sistema que genera y valida circuitos analógicos CA/CD a nivel de componentes comerciales, con la simulación SPICE incorporada dentro del lazo de diseño. El diseño se articula en tres planos que se sostienen entre sí. La descomposición funcional y los casos de uso (sección 4.3) establecen qué hace el sistema y quién interactúa con él. El modelo C4 en sus tres niveles (sección 4.4) establece cómo se estructura: la separación en subsistema cliente y subsistema servidor, y la organización del servidor como un ecosistema de agentes modulares —Orquestador, Cálculo, Curador, Shell y Escritura— coordinados sobre un grafo con estado y persistencia. El diagrama de secuencia (sección 4.5) establece cómo se comportan esos elementos en el tiempo, integrando el lazo de ajuste iterativo que distingue a la propuesta.

Los criterios de aceptación enunciados en (sección 4.6) fijan la condición que el sistema construido deberá cumplir. Su comprobación —la medición del comportamiento del sistema frente a esos criterios sobre un conjunto de circuitos de prueba— constituye la evaluación del sistema y se desarrolla en el capítulo siguiente, que toma este diseño como objeto de medida.

Capítulo 5

Pruebas y Análisis de Resultados

Capítulo 6

Conclusiones y Trabajo Futuro

Referencias

- [1] Z. Bao et al. «AnalogAgent: Self-Improving Analog Circuit Design Automation with LLM Agents.» arXiv: 2603.23910 [cs.AI]. dirección: <https://arxiv.org/abs/2603.23910>.
- [2] H. Zou, P. Han, E. Nazerian y A. Q. Huang. «PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards.» arXiv: 2602.00510 [cs.SE]. dirección: <https://arxiv.org/abs/2602.00510>.
- [3] K. S. Al Hasan, S. R. Raiyan, H. M. Alvee y W. Sadik. «CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts.» arXiv: 2601.04505 [cs.SE]. dirección: <https://arxiv.org/abs/2601.04505>.
- [4] Y. Lai et al., «AnalogCoder: Analog Circuit Design via Training-Free Code Generation,» en *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence (AAAI-25)*, 2025.
- [5] Z. Wang, J. Gao, W. Fu, X. Guo y X. Zhang. «AnalogSAGE: Self-evolving Analog Design Multi-Agents with Stratified Memory and Grounded Experience.» arXiv: 2512.22435 [cs.AR]. dirección: <https://arxiv.org/abs/2512.22435>.
- [6] W. Qiu, Y. Xiao y R. Pan. «HWE-Bench: Can Language Models Perform Board-level Schematic Designs?» arXiv: 2603.18102 [cs.SE]. dirección: <https://arxiv.org/abs/2603.18102>.
- [7] Ngspice Development Team, *Ngspice User's Manual*, Version 44. dirección: <https://ngspice.sourceforge.io/docs/ngspice-manual.pdf>.
- [8] LangChain. «LangGraph: Building stateful, multi-actor applications with LLMs. »dirección: <https://github.com/langchain-ai/langgraph>.
- [9] F. Salvaire, *PySpice: Simulate electronic circuit using Python and the Ngspice/Xyce simulators*, 2021. dirección: <https://github.com/PySpice-org/PySpice>.
- [10] Y. Yin, Y. Wang, B. Xu y P. Li. «ADO-LLM: Analog Design Bayesian Optimization with In-Context Learning of Large Language Models.» arXiv: 2406.18770 [cs.SE]. dirección: <https://arxiv.org/abs/2406.18770>.

- [11] D. Vungarala, S. Alam, A. Ghosh y S. Angizi. «SPICEPilot: Navigating SPICE Code Generation and Simulation with AI Guidance.» arXiv: 2410.20553 [cs.SE]. dirección: <https://arxiv.org/abs/2410.20553>.

Apéndice A

Diagramas de casos de uso

Los siguientes diagramas complementan el modelo de casos de uso presentado en la sección 4.3.2. Cada uno detalla un subconjunto de interacciones del sistema.

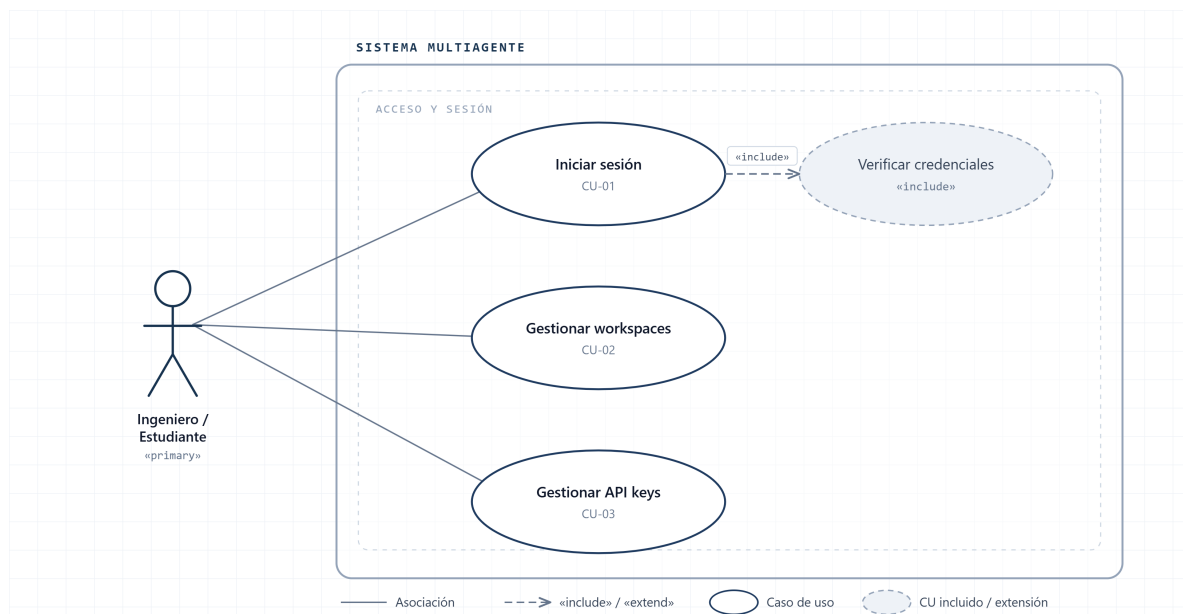


Figura A.1. Diagrama de casos de uso: acceso y gestión de sesiones.

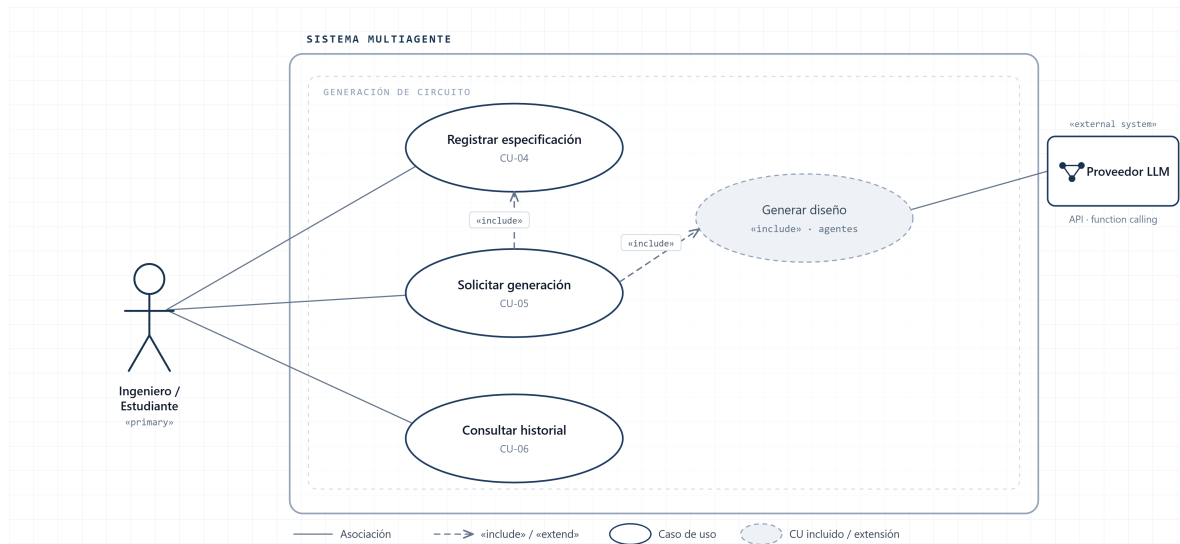


Figura A.2. Diagrama de casos de uso: generación de circuito.

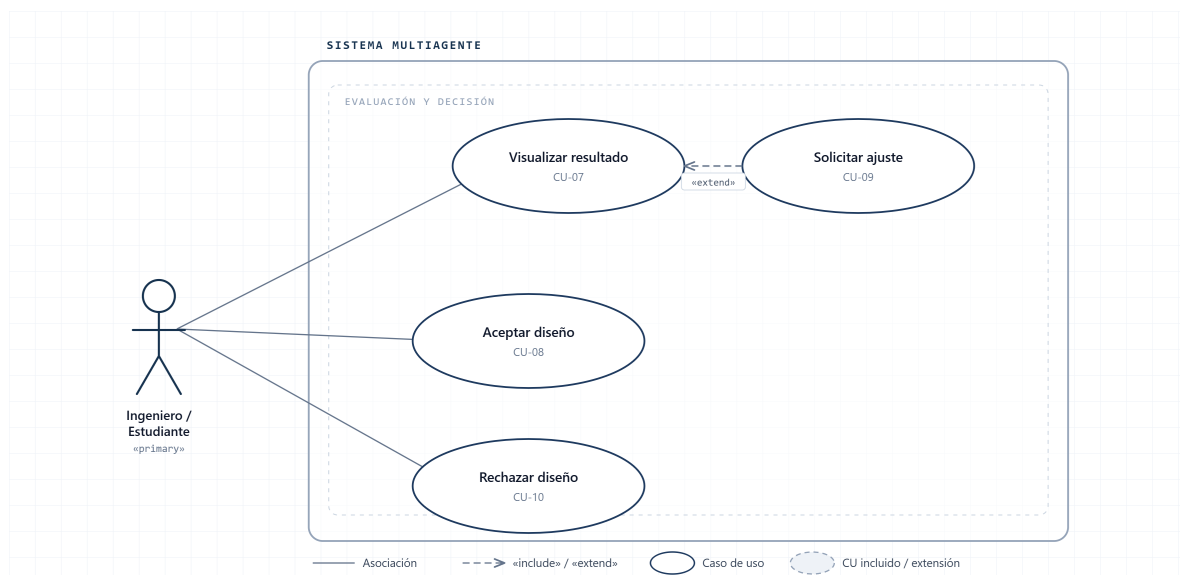


Figura A.3. Diagrama de casos de uso: decisión del agente curador.